

서브쿼리(하위 질의)

ch07.sql 코드를 실행 순서대로 분석한 학습자료입니다. HR 스키마 (EMPLOYEES/DEPARTMENTS) 기준입니다.

07

01 단일행 서브쿼리 — 결과가 1개일 때만 가능

```
-- 'De Haan'과 같은 급여를 받는 직원 (De Haan은 결과가 1명 → 정상 동작)
select * from employees
where salary = (select salary from employees where last_name = 'De Haan');

-- ❌ 에러 발생 예제: 'Taylor'가 2명 이상이면 서브쿼리가 값을 2개 이상 반환
-- → "single-row subquery returns more than one row" 오류
select * from employees
where salary = (select salary from employees where last_name = 'Taylor');
```

단일행 연산자(=, >, < 등)의 규칙 — 서브쿼리 결과가 반드시 **정확히 1행**이어야 합니다. 몇 개가 나올지 확신할 수 없다면 IN 같은 다중행 연산자로 바뀌어야 합니다.

02 다중행 연산자 — IN

```
-- 리터럴 값 목록으로 직접 조회 (참고용)
select * from employees where salary in (3200, 8600);

-- 'Taylor'라는 성을 가진 사람이 여러 명이어도 안전하게 동작 (다중행 반환 허용)
select * from employees
where salary in (select salary from employees where last_name = 'Taylor');
```

7-1과 비교 — 같은 상황(Taylor 여러 명)이라도 = 대신 IN을 쓰면 에러 없이 정상 처리됩니다. "서브쿼리 결과 개수가 불확실하면 IN"이 안전한 기본 전략입니다.

03 다중 열(컬럼) 서브쿼리

괄호로 여러 컬럼을 묶어 서브쿼리의 여러 컬럼 조합과 동시에 비교합니다. "job_id별 최저 급여자"처럼 그룹별 조건을 걸 때 유용합니다.

```
-- job_id별로 가장 작은 급여를 받는 사람 조회
select first_name, job_id, salary
from employees
where (job_id, salary) in (
    select job_id, min(salary)
    from employees
    group by job_id
)
order by salary;
```

구조 분석 — 서브쿼리는 (job_id, 그 직군의 최저급여) 쌍의 목록을 만들고, 바깥쿼리는 (job_id, salary) 조합이 그 목록에 있는 행만 골라냅니다. GROUP BY + 다중열 IN의 대표 조합 패턴입니다.

04 인라인 뷰(Inline View)

FROM절에 등장하는 서브쿼리를 인라인 뷰라 하며, 결과가 마치 하나의 임시 테이블처럼 취급되어 다른 테이블과 조인할 수 있습니다.

```
select *
from employees A, (
    select department_id
    from departments
    where department_name = 'IT'
) B
where A.department_id = B.department_id;
```

ch03 도입 예제와 연결 — 1강에서 예고했던 "괄호 안 SELECT가 테이블처럼 쓰이는" 구조가 바로 이것입니다. B는 IT 부서의 department_id 하나만 담은 임시 테이블 역할을 합니다.